Machine Learning Project on Formula 1

Achour Myriam, Berrebi Solal

May 26, 2025

Contents

1 Search and construction of the DataSet **2**
1.1 First Approach: Manual Web Scraping . . . 2
1.2 Second Attempt: Extraction from GitHub (`f1db`) . . . 2
1.3 Final Attempt: Kaggle's dataset . . . 2
1.4 Parallel Approach Considered: FastF1 API . . . 3

2 Analysis of the dataset and cleaning **3**
2.1 Some analysis to see if the dataset is correct . . . 3
2.2 Cleaning the Dataset . . . 5

3 Unsupervised learning **7**
3.1 Clustering of strategies . . . 7
3.1.1 Limitation of this approach . . . 10
3.2 Clustering circuits . . . 10
3.2.1 First attempt . . . 11
3.2.2 Second attempt : FastF1 API . . . 11
3.2.3 Last attempt : FastF1 API . . . 12
3.2.4 Conclusion and link with previous section . . . 16

4 Supervised learning **16**
4.1 Prediction of Championship Points: Results and Discussion . . . 17
4.2 Prediction of the Full Grid . . . 18
4.2.1 Predicting the Full Grid with our dataset . . . 18
4.2.2 Predicting the Full Grid with FastF1 API . . . 20
4.3 Pit Stop Prediction - Modeling Pit Stop Counts with Meteorological Data 21
4.4 Bonus task : Classification of Sentiments . . . 23
4.4.1 The dataset . . . 23
4.4.2 Preprocessing Data . . . 23
4.4.3 Training and testing models . . . 24
4.4.4 Predicting on real data . . . 24
4.4.5 Last task . . . 25

5 Bibliography **26**

Introduction

Formula 1 is one of the sports where data plays a crucial role, whether it is to estimate the best strategy to cross the finish line or adjusting the pilot's approach during a race in real time. We found this theme particularly interesting, which is why we decided to choose it for our machine learning project.

Our primary goal is to predict the number of points each pilot will score in a given race, as well as to forecast the complete finishing grid. Additionally, we conducted exploratory data analysis, including clustering pilot strategies across different circuits, classifying circuits based on various criteria, and more.

Race engineers, alongside car engineers, are responsible for optimizing a pilot's performance. To achieve this, they employ a range of techniques to determine the best possible strategy. These include strategy simulation, tire model optimization, free air analysis, and Monte Carlo simulations to predict potential race outcomes. This process is just one aspect of the extensive engineering efforts behind finding the ideal strategy for a pilot to win a race.

1 Search and construction of the DataSet

One of the most valuable resources for engineers is data. In our search for the ideal dataset, we explored multiple approaches to find one that was both comprehensive and challenging to use, unlike the more accessible FastF1 API. This initial step proved to be quite a struggle, as we experimented with various methods to identify the best dataset for our needs.

1.1 First Approach: Manual Web Scraping

The first idea was to build a dataset using web scraping on sites like `formula1.com` or `ergast.com`. However, this approach quickly proved difficult to implement effectively, due to the complexity of HTML structures and the anti-scraping protections in place.

1.2 Second Attempt: Extraction from GitHub (`f1db`)

Attention then turned to a GitHub repository titled f1db, which contains Formula 1 data in YAML files. This repository seemed promising as it gathers multiple seasons of F1 with an interesting level of detail (information on drivers, teams, circuits, results, etc.).

We decided to retrieve a restricted portion of the data, as the dataset took too long to download. So we downloaded those associated with the 2015–2025 F1 seasons from the repository **f1db**. By developing a Python script inspired by a response on the Stack Overflow forum[1], we were able to extract relevant features from the *race.yml* and *result-race.yml* files. As we were unsure of the quality of the data we preferred to turn ourselves to a more reliable and complete one.

1.3 Final Attempt: Kaggle's dataset

We finally decided to use a Kaggle dataset the F1 GrandPrix Dataset as it was more reliable in terms of quality and provided access to comprehensive data. We therefore made

a Python script to gather all the most important data in a single csv file **f1_dataset.csv**. We :

- converted some times in seconds
- created derived columns from the others such as : *num_pit_stops* to count the number of pitstops made by a driver during a race and *total_pit_time_ms* to retrieve the total pitstop time of a pilot during a race

Each row represents a driver in a Grand Prix race, including details such as qualifying results (*q1, q2, q3*) (qualifying times), the number of laps raced, pit stop count, fastest lap speed, lap time, and more.

1.4 Parallel Approach Considered: FastF1 API

In parallel, an exploration of the Python library `FastF1` was even so initiated. This library provides direct access to official F1 data via the API of `formula1.com`. The advantage of this approach is twofold:

- Extraction is simple.
- The data is structured in objects similar to `DataFrame`, easily convertible into CSV.

However, we only used it to compare the results of our models using our dataset and the FastF1 API, or we relied on the API when our dataset lacked sufficient data to perform certain tasks.

2 Analysis of the dataset and cleaning

We already knew how important the quality of a dataset was, to work with it and make prediction or different machine learning tasks. However, throughout this project, we gained an even deeper understanding of the principle "garbage in, garbage out", reinforcing how crucial data cleaning and thorough analysis are in achieving meaningful and reliable results.

2.1 Some analysis to see if the dataset is correct

Thus, we decided to make exploratory analysis on our dataset.

First, we identified the top 10 drivers of all times. The following bar plot shows the result :

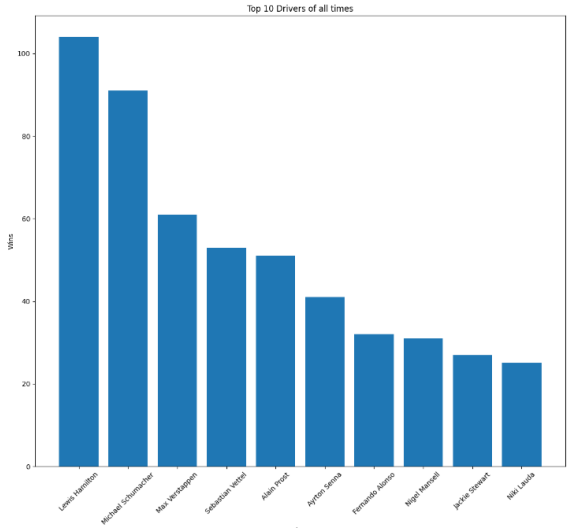


Figure 1: Top 10 drivers of all times

The top 3 being Hamilton, Schumacher and Verstappen is correct.

We also plotted the 10 best constructors of all times :

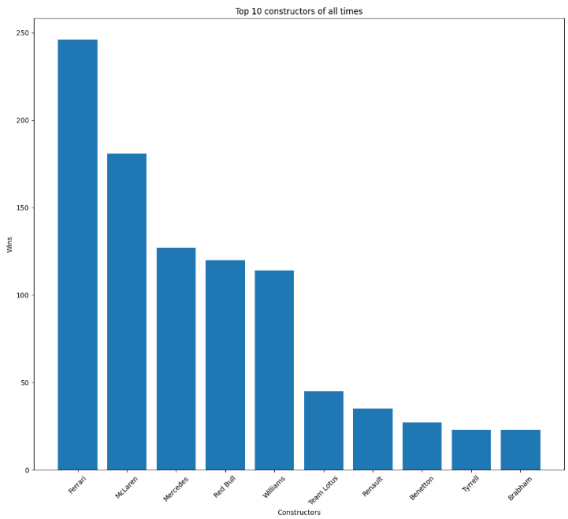


Figure 2: Top 10 constructors of all times

The top 3 being Ferrari, McLaren followed by Mercedes is also correct !
The dataset seemed right.
We then plotted the distribution of drivers' final positions across all races and obtained the following result :

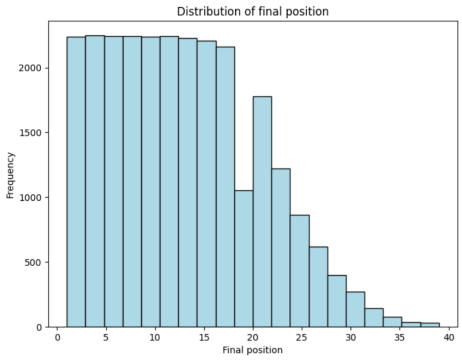


Figure 3: Distribution of final position of drivers in the dataset

We see that many positions are between 1 and 24. It corresponds to the fact that in Formula 1 nowadays there are often 20 drivers competing. But still we did not understand why some positions where over 30 and more. We, hence, ran these lines :

```
#Abnormal data in the dataset
df_abnormal = df[df["positionOrder"] > 30]
list_abnormal_years=[int(x) for x in df_abnormal["year"].unique()]
print("Years with many people on a same circuit are:", list_abnormal_years)
```

And we obtained that between 1950 and 1992 some races hosted more than 30 drivers, which is historically accurate ! Here is an example : italian GP 1990

Finally, we decided to see if we could find relations between features. We obtained the following correlation matrix :

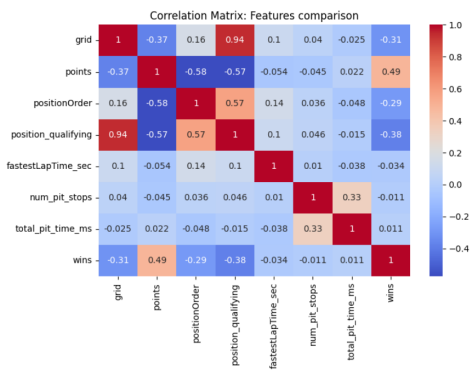


Figure 4: Correlation matrix before cleaning on relevant features

We observe, for instance, that the higher the final position (positionOrder) of a driver is, i.e the worse its position is, the smaller is its number of points, which is completely logical, as points depend on the final position.

2.2 Cleaning the Dataset

Initially, we attempted to work directly with the dataset, but we soon realized that this approach was not optimal, as our results were really disappointing.

And after running this set of lines :

```
df.replace( to_replace: "\\N", np.nan, inplace=True)
df.replace(-1, np.nan, inplace=True)
df_null_info=df.isnull().sum()
print(df_null_info[df_null_info>10000])
```

We saw that many, too many columns were filled with meaningless values.

Moreover some pits stops time seemed quite high or regarding the fact that many lines were mostly empty we wondered on how many drivers actually finished their race. So we plotted an histogram of the final status of races for each driver :

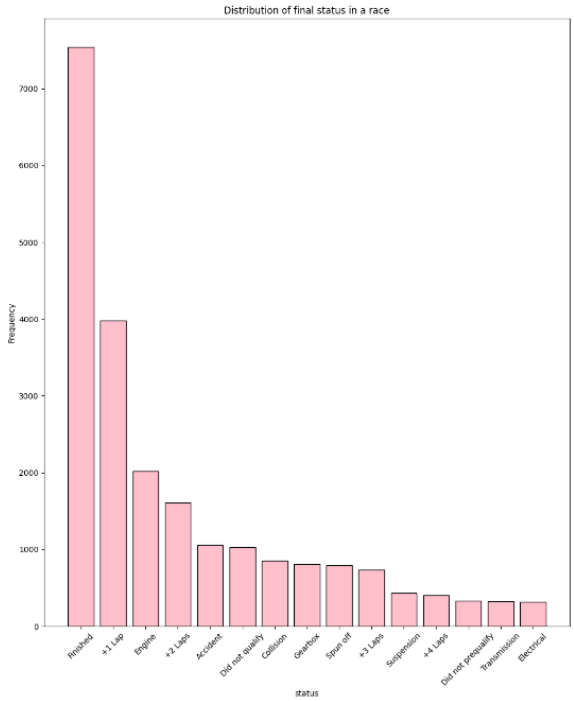


Figure 5: Distribution of final status of drivers during races in the dataset

We can observe that around 7500 lines indicate a finished race and around 7000 lines correspond to people not finishing with the same number of laps (+1 lap, +2 laps, +3 laps,..., $+x$ laps) a given race but still being ranked. The others did not complete their race. This means that more than half of our dataset of approximately 27000 lines is filled with lines of drivers not completing a race, having an accident or any other problems. Which helped us understand why so many lines where filled with wrong data or almost empty.

So we ran this line to remove the rows that were skewing the results :

df = df.dropna(thresh= len(df.columns) * 0.7)

This line remove rows containing more than 70% of NaN slots. Why 70%? Because sometimes some columns are not filled even if the driver completed a race, such as q2 and q3. These represent the qualifying times of drivers in the second and third qualifying sessions, which are important for determining the starting grid. Drivers compete in round 1 (q1); some are eliminated from progressing to the next rounds but can still participate in the final race, so they won't have q2 or q3 times. Choosing a threshold below 70% could have led to the removal of useful data, so we decided to set the bar relatively high Still it drastically reduced the number of rows of our dataset but it improved significantly our results and analysis.

Following this preprocessing step, we decided to see if the relations between features could be better figured out. And indeed as we can see on this correlation matrix :

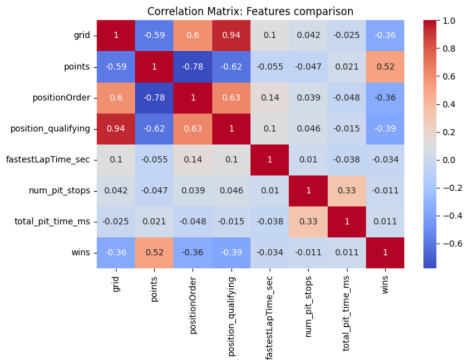


Figure 6: Correlation matrix, after cleaning, between relevant features

Some correlation coefficients have slightly improved. We can point out the one between grid and points. Moreover, we can now clearly see that the higher the starting position of a driver (grid), the higher will be his end position (positionOrder). Reinforcing the impact of qualification results on the outcome of the actual race. *Note : "Winning" qualifications allow to have a better starting position for the race.*

After this in-depth analysis, we restarted doing our tasks. First, let's talk about our unsupervised learning tasks.

3 Unsupervised learning

3.1 Clustering of strategies

One thing we found interesting was to analyze how different drivers' strategies vary across different circuits.

Here, we identify a strategy based on :

- the number of pit stops made during a race
- the time spent at the pit
- the speed during the fastest lap
- the time of the fastest lap

We consider the best strategy to be the one that enables a driver to maintain or significantly improve their position during a race.

Therefore, we clustered drivers for four different circuits in four different countries to have a broad overview of possible strategy variations.

We chose to cluster races from 2015 to 2024, ensuring that we had as much data as possible. As explained earlier, before the 2000s, various issues affected race conditions, skewing the dataset.

How did we approach the problem ?

Our goal was to apply **K-Means algorithm** on our dataset for each circuit.

We decided to consider only drivers who actually finished their race (with status==1). *Not finishing a race on purpose cannot be considered as a strategy.*

First, we started by plotting data, running a pca beforehand to try distinguishing how many clusters we could have in the dataset and then choose the right k to apply K-Means algorithm. We then normalized the data, applied the algorithm, but initially obtained clusters that did not meet our expectations.

Following further research, we found an article[2] explaining the Elbow Method, which helps determine the most suitable k. Although the curves did not show a distinct break, we identified a points that resembled an "elbow", selecting it as the optimal k. After implementing this, the clusters became significantly clearer.

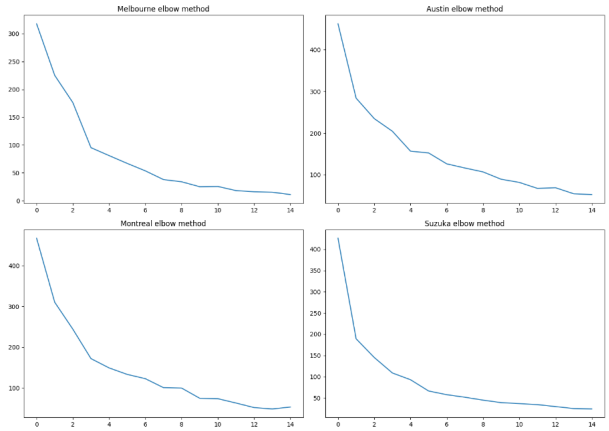


Figure 7: Plot of the elbow method applied to 4 circuits

We can see for Melbourne an elbow at $k = 3$, for Austin one at $k = 4$, for Montreal at $k = 3$ and for Suzuka at $k = 5$.

We followed quite the same method as this article [3] but for our task which is quite different as we clustered drivers over 10 years on a circuit and not drivers over a given period of time. As suggested in the article, to better understand strategies within each cluster, we plotted randomly selected drivers for each group.

A new problem occurred !

During our analysis, we identified unexpectedly high pit stop times, exceeding 1,000,000 ms. After thorough investigation, we confirmed that this issue was not due to errors in the dataset or data conversion on our part. Instead, it comes from race incidents (i.e red flags, safety car) where drivers were forced to stop for prolonged periods, leading to recorded pit stop times exceeding 20 minutes or more, as observed in this particular race.Hungarian GP 2021.

Quick note : *Here for instance many people had to stop at lap 2 because of an accident leading to a red flag, this for about 24 minutes !*

So we decided to trim these outliers, as these long times of stops are not a part of a strategy but due to a random event skewing the results.

After this adjustment we obtained the following clusters, plotted using a pca :

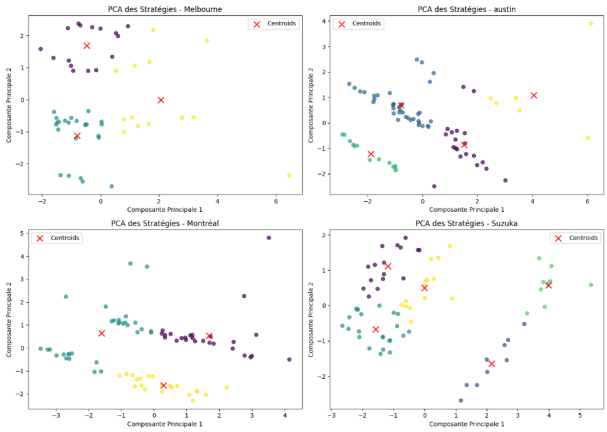


Figure 8: We can see 3 clusters in Melbourne, 4 for Austin, 3 for Montreal and 5 for Suzuka

What do we observe ? The strategies implemented on each circuit vary significantly, with clusters located in different regions for each track, reflecting diverse racing approaches.

For Melbourne : Drivers in all clusters at Melbourne recorded fastest lap speeds between 215 and 230 km/h. They made at most two pit stops with total stop times under 40 seconds on average. The best strategy combines high speed with few pit stops.

The best strategy seems to be the following :

```
Some drivers in cluster 1 are :
       forename surname  grid  year  positionOrder  total_pit_time_ms  num_pit_stops  fastestLapTime_sec  fastestLapSpeed
25444     Lando  Norris     4  2022              5            17556.0            1.0              82.248          231.018
26327  Fernando  Alonso    10  2024              8            36155.0            2.0              80.493          236.055
```

The silhouette score obtained for this clustering is : 0.34

For Austin : Compared to Melbourne drivers adopt a slower pace as their fastest lap speed is between 190 and 200km/h. They also stop more frequently (at least 2 times) and for a longer time (more than 45sec).

The best strategy is quite different as it seems to be the following, using a slower pace :

```
Some drivers in cluster 3 are :
      forename     surname  grid  year  positionOrder  total_pit_time_ms  num_pit_stops  fastestLapTime_sec  fastestLapSpeed
25761    Lewis    Hamilton     3  2022              2            48497.0            2.0              99.830          198.805
25280      Max  Verstappen     1  2021              1            47779.0            2.0              99.096          200.278
```

The silhouette score obtained for this clustering is : 0.23

For Montreal : We observe that in every cluster drivers fastest lap speed is on average at 200km/h, they stop one or two times and for less time than in Austin.

The best strategy seems to be a mix between an average pace and few stops :

```
Some drivers in cluster 2 are :
      forename     surname  grid  year  positionOrder  total_pit_time_ms  num_pit_stops  fastestLapTime_sec  fastestLapSpeed
24324      Max  Verstappen     9  2019              5            23426.0            1.0              74.767          209.980
25985   Sergio       Pérez    12  2023              6            46499.0            2.0              74.481          210.786
```

The silhouette score obtained for this clustering is : 0.27

Finally, for Suzuka : We observe that in every cluster drivers fastest lap speed is on average at 215km/h and they stop two or three times, for a total pit stop time of more than 46sec.

The best strategy being going at high speed making at least two longer pit stops

```
Some drivers in cluster 2 are :
       forename    surname  grid  year  positionOrder  total_pit_time_ms  num_pit_stops  fastestLapTime_sec  fastestLapSpeed
26142     Oscar    Piastri     2  2023              3            47653.0            2.0              96.328          217.021
23268      Kimi  Räikkönen     8  2016              5            46006.0            2.0              95.990          217.785
```

The silhouette score obtained for this clustering is : 0.39

Four countries, four circuits and four different strategies. It is interesting to see that just with some features we can already detect different patterns in drivers' driving strategy according to the place they drive in and how throughout the years we can clearly see that drivers tend to adopt the same strategy (as we can distinguish clusters, even some overlap), and that there are just a few outliers.

In conclusion, we can summarize the results like this :

Circuits	N°Clusters	Avg Speed	Pit Stop	Best strategy
`Melbourne`	3	220km/h	1-2	Agressive : Fast + few stops
`Austin`	4	190-200km/h	2-3	Conservative : Slow + Many stops
`Montreal`	3	200km/h	1-2	Moderate : Medium speed + few stops
`Suzuka`	5	215km/h	2-3	Fast but still many stops (likely due to tire damage from many corners maybe, see next section)

3.1.1 Limitation of this approach

The clusters detected by the algorithm are not exactly the same each time as the initialisation for K-Means is random.

The inertia of the clusters seems quite high and the silhouette score remains low but it is the problem with analyzing this kind of data, containing lots of noise, clusters overlap a bit. We could possibly further improve the model. It is something to think more about.

3.2 Clustering circuits

This led us to a new question :

Why can drivers' strategies differ significantly between some circuits, while remaining similar on others?

This is most likely due to the layout of the circuit they are in. A large number of corners may lead to more pitstops due to increased tire damage for instance.

Therefore, to understand why driver strategies differ by circuit, we clustered circuits themselves.

We decided to work on the Grand Prix of 2023. It contains 22 circuits GP 2023

The features on which we decided to cluster being the followings :

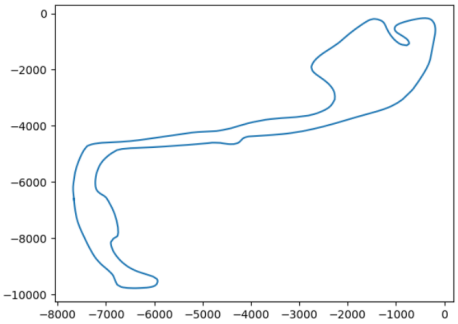


Figure 9: Monaco circuit

1. Length :*length*
2. The average speed of a given driver (We chose Max Verstappen as he was most likely to finish every race): *bestAvgSpeed*
3. The number of corners it contains : *Note_corners*
4. The number of corners being hairpins : *Hairpin*
5. The number of corners being sharp : *Sharp*

3.2.1 First attempt

Firstly, we started by using the following dataset Circuits on GitHub. It is a dataset containing the list of circuits in which the drivers competed during a given year. This data set contains *geojson* data with coordinates of points describing the circuit being in latitude and longitude, after trying different method to convert them to a cartesian referential, and we quickly gave up... And turned to the FastF1 API.

3.2.2 Second attempt : FastF1 API

We then found that on the FastF1 API it was much easier to draw a circuit and have access to the coordinates of its points. We then had to determine the value of each feature for each track. We wanted to compute the angles of the corners of each circuits by ourselves to then classify them using a function we created based on our assumption of the appropriate thresholds to classify an angle by sharpness.

$$\begin{aligned}
&\text{If } 0 < \hat{\theta} < 20^\circ \text{ or } -20^\circ < \hat{\theta} < 0, && \Rightarrow \text{ Hairpin} \\
&\text{Else if } 20^\circ \leq \hat{\theta} < 125^\circ \text{ or } -125^\circ \leq \hat{\theta} < -20^\circ, && \Rightarrow \text{ Sharp angle} \\
&\text{Else if } 125^\circ \leq \hat{\theta} \leq 170^\circ, && \Rightarrow \text{ Medium angle}
\end{aligned}$$

To compute the angle between three points, we proceeded like this :
Given three points : $\mathbf{a}$, $\mathbf{b}$, $\mathbf{c}$

Let's define the vectors:

$$\mathbf{ba} = \mathbf{a} - \mathbf{b}, \quad \mathbf{bc} = \mathbf{c} - \mathbf{b}$$

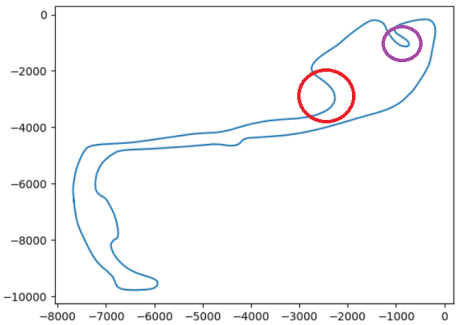
And computed the angle in radians:

$$\theta = \arccos\left(\frac{<\mathbf{ba}, \mathbf{bc}>}{\|\mathbf{ba}\|\|\mathbf{bc}\|}\right)$$

Finally, converting it into degrees:

$$\hat{\theta} = \theta \times \frac{180}{\pi}$$

To apply these formulas correctly, we had to carefully choose the points a, b, and c. As it could have been expected beforehand these points depends a lot on the corresponding corner we work on. Indeed, take a look at Monaco circuit again :



To compute the angle inside the red circle we need points that are farther from each other than for the purple circle.

So, we had the idea to try detecting the angles by applying our method in several pass :

1. First for sets of three points separated by 5 points from each other, to detect shape of smaller angles.

2. then repeating for points separated by 10 points from each other to detect medium angles.

3. then for points separated by 20 points from each other, to detect wider angles.

   It seemed to work a bit but when we compared with the real structure of circuits we noticed that some corners were missed while others were incorrectly detected (false positives). The results were too noisy to be reliable.

3.2.3 Last attempt : FastF1 API

We finally decided to use a method of FastF1 API which directly provides the number of corners along with their corresponding angles using the line *session.get_circuit_info().corners*, to get the information of each corner of a circuit. After that we just had to apply the function to classify each corner to proceed to the clustering.

To apply K-Means algorithm, we once again normalized data. Then we visualized data using a pca :

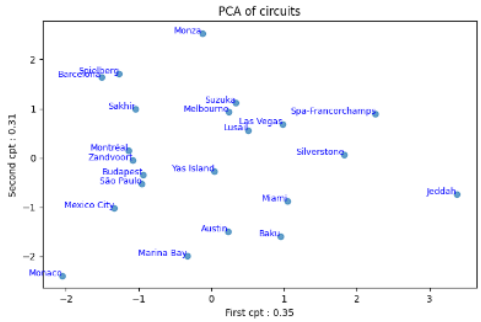


Figure 10: PCA of circuits of 2023 F1 Grand Prix

Before applying a hierarchical clustering we decided to use the elbow method to see the optimal number of clusters we could need and we obtain the following figure :

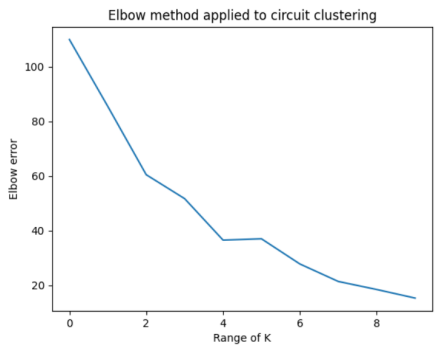


Figure 11: Elbow method for circuits clustering

We can see a small elbow at $k = 4$ so the optimal number of clusters should be around 4, as this minimizes inertia.

Then we applied hierarchical clustering using Ward's method in scipy.linkage, which minimizes the variance inside each cluster. We obtained the following clustering :

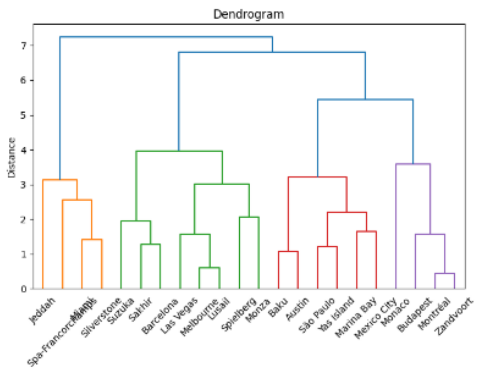


Figure 12: Hierarchical clustering of circuits of 2023 session

We can identify 4 main groups :

The first 1, in orange, being the one containing circuits having difficult corners, in particular hairpins but which are still manageable as drivers can drive there at a high speed, at an average of 230 km/h, thanks to the straight lines they have.

```
Some circuits in cluster 0 are :
               Circuit  Hairpin  Sharp  Nb_corners  bestAvgSpeed  length
9          Silverstone        4     10          18    230.999419   5.891
1               Jeddah        4      9          27    238.167640   6.174
11  Spa-Francorchamps        3      7          19    228.317622   7.004
```

We can cite the two followings

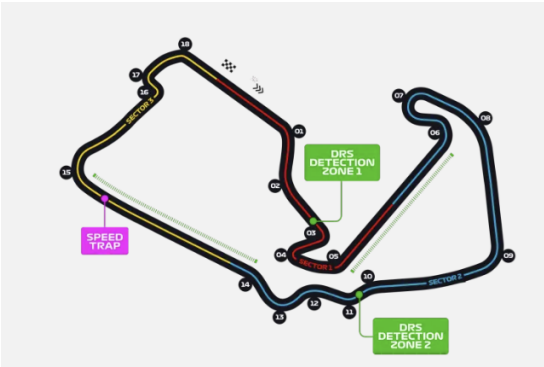


Figure 13: Silverstone, Great Britain

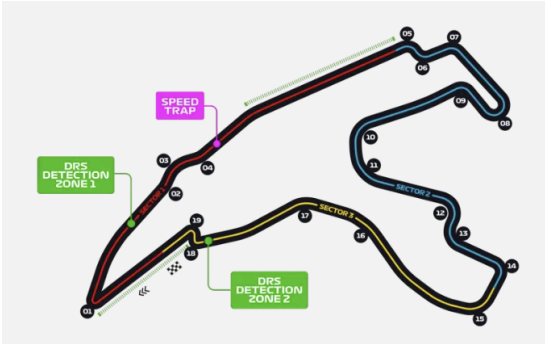


Figure 14: Spa-Francorchamps, Belgium

The second one could be the green one. It corresponds to circuits having a relatively small number of corners (except Suzuka having 18, the others have around 10-11 or 14), in particular not really tight ones and allowing drivers to go at really high speeds.

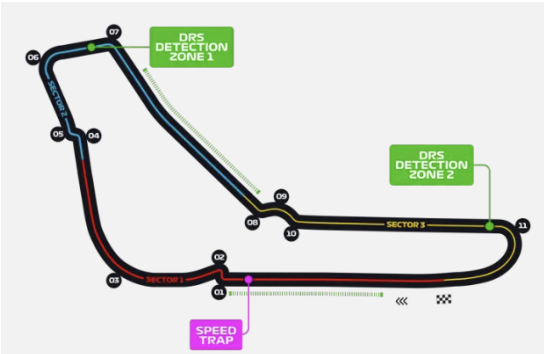


Figure 15: Monza - Temple of speed, Italia

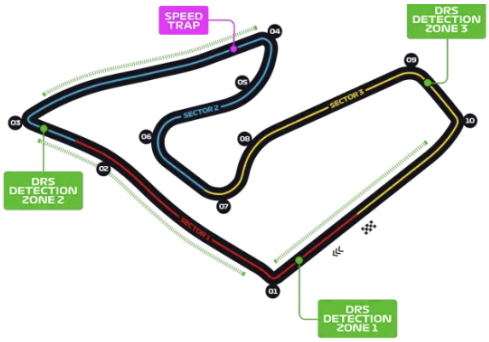


Figure 16: Red Bull Ring - formerly Spielberg, Austria

The third cluster, shown in red, includes circuits with a high number of corners (typically 17 to 20). These layouts reduce average speeds significantly, often keeping them below the 200 km/h threshold, due to the technical nature of the tracks.

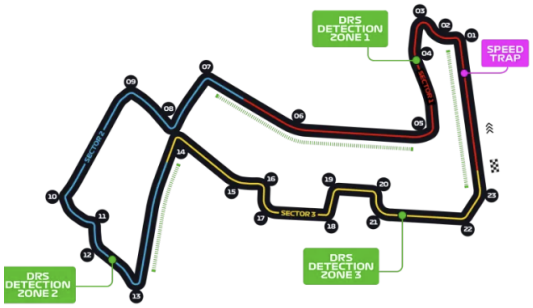


Figure 17: Marina Bay, Singapore

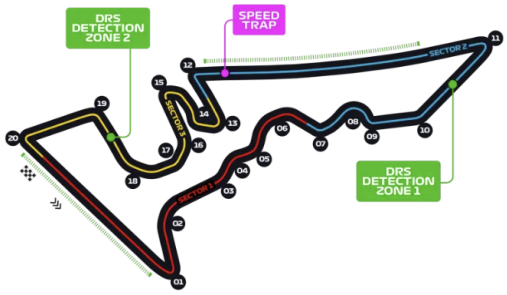


Figure 18: Austin, United States of America

Some circuits in cluster 2 are :

	Circuit	Hairpin	Sharp	Nb_corners	bestAvgSpeed	length
17	Austin	0	12	20	196.259778	5.513
3	Baku	0	14	20	204.526961	6.003
14	Marina Bay	1	12	19	180.644807	5.063

And finally we can talk about the last cluster in purple. It is formed with small circuits (3.33 to 4.3 km against 4.3-7 for the others), having sharp corners (2-3 hairpins) and not allowing really high speeds (210 km/h at most)

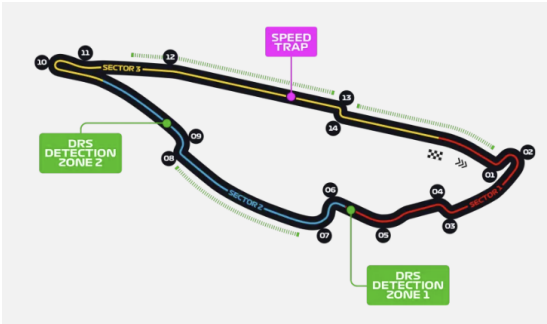


Figure 19: Gilles-Villeneuve, Canada

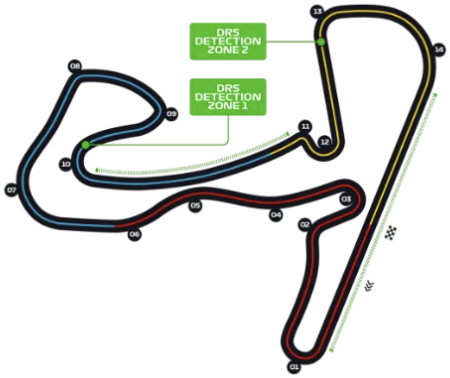


Figure 20: Zandvoort, Netherlands

We can also point out one thing that the algorithm detects well. Monaco is a one-of-kind circuit. It is in the last cluster yes but added really late, as we can see on the dendrogram. It is because it is really small, 3.337km, have really hard corners to take especially the hairpin of loewe n°6, taken in average at a speed of 45km/h. Indeed, Monaco's circuit is the lowest speed one and here the average speed estimated with our script for this circuit is just at 154 km/h !

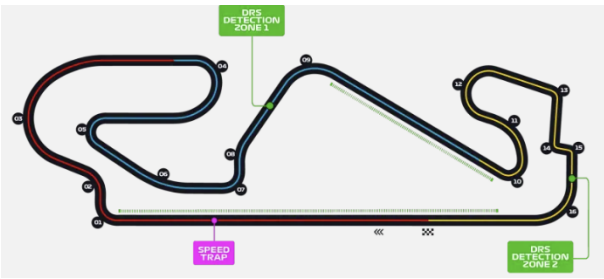


Figure 21: Monaco, Monaco

3.2.4 Conclusion and link with previous section

We can now understand why drivers have to stop more in Austin circuit and why they do not drive at really high speed. It is mainly due to the complicated layout of this circuit, many corners and sharp ones in particular ! In Melbourne, which is in cluster 2 they can go fast because there are not a lot of corners, allowing drivers to go faster. And we saw earlier that drivers stopped a bit more in Suzuka than in Melbourne in average (3 times against 2 in average) but still had a high speed, more than 215km/h, and indeed Suzuka belongs to cluster 2 because its corners are not that sharp only 6 out of 18 are tight corners. And drivers may have to stop more because still 18 corners might damage tire faster. In Montreal we saw that drivers drive at around 200km/h and Montreal belongs to the fourth cluster, which seems coherent as the circuit there are not really high speed circuits.

Altogether, this analysis shows how circuit layout strongly influences the pit-stop strategy and the overall performance of all players.

Note : Limitations We observe that sometimes the best k using the elbow method can be 5, so we computed the inertia in the case of 5 clusters and 4 clusters as well as the resulting silhouette score. The silhouette score indicates the quality of the clustering. If it is closer to 0, some clusters are mixed if it is close to 1 then the clustering is efficient.

For $k = 4$, the inertia is of 50 and the silhouette score of 0.22 For $k = 5$, the inertia is of 36.6 and the silhouette score of 0.28

This aligns with the fact that some circuits remain quite similar in their design, making it difficult to have a perfect clustering.

These scores are also subject to change according to the initialisation of the K-Means algorithm.

4 Supervised learning

Supervised learning methods lie at the heart of predictive analytics in Formula 1. Unlike unsupervised approaches that aim to uncover hidden patterns in the data, supervised learning relies on labeled examples, inputs paired with known outputs, to train models that can make accurate predictions on new data. In the world of F1, where each race generates thousands of data points (qualifying times, telemetry, weather conditions, pit stop strategies, and more), these methods are essential to predict performances and behaviors of the drivers which if not apprehended can cost millions of dollars.

4.1 Prediction of Championship Points: Results and Discussion

1. Data preparation and feature engineering.

- **Target:** total points scored by a driver in a race.
- **Features:** qualifying times (Q1–Q3 in seconds), grid position, number of pit-stops, total pit-stop time, fastest lap time, prior championship ranking, circuit and constructor IDs.
- **Preprocessing:** missing Q times imputed to session median; numerical features mean-imputed and standardized; categorical features imputed to "NA" and one-hot encoded.

2. Model selection and hyperparameter search. Six regressors were compared:

- *Linear models:* Ridge, SGD Regressor
- *Kernel methods:* KNN, SVR
- *Tree ensembles:* Random Forest, Gradient Boosting

Each model was tuned via `RandomizedSearchCV` (5-fold, MAE scoring) to optimize key hyperparameters.

Cross-validation. To obtain a reliable estimate of our models' generalization performance and to mitigate overfitting, we employed K-fold cross-validation on the training set. In this procedure, the data is partitioned into K equally sized folds (we chose $K = 5$). For each of the K iterations, one fold serves as the validation set while the remaining $K-1$ folds are used for training; the process is repeated so that every fold is used exactly once for validation. The model's cross-validated score is then taken as the average performance across all folds. In our experiments, we applied 5-fold cross-validation within the hyperparameter search (RandomizedSearchCV) using mean absolute error (MAE) as the scoring metric. This helps us choose hyperparameters based on consistent performance rather than the quirks of a single train/test split.

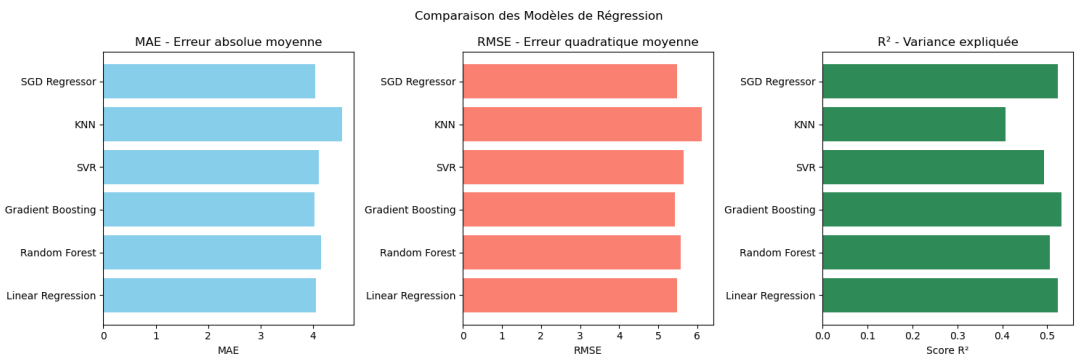


Figure 22: True vs. predicted points for the top three models (GBR, Linear Regression, SGD).

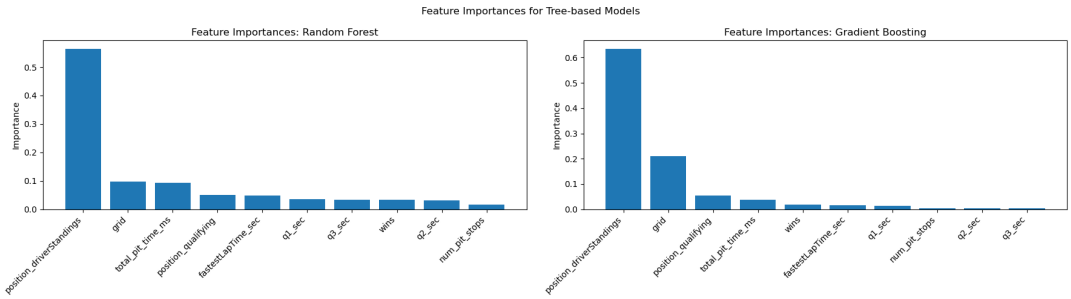


Figure 23: Feature importance for Random Forest (left) and Gradient Boosting (right).

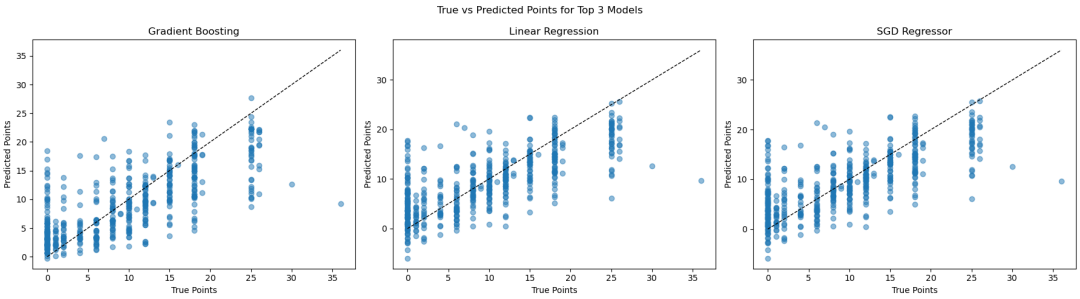


Figure 24: Comparison of MAE, RMSE and R^2 across all six models.

Key takeaways.

- **Best performers:** Tree-based ensembles (GBR and RF) achieve the lowest MAE ($\approx$4 points) and highest R^2 ($\approx$0.55).
- **Linear models competitive:** Ridge and SGD reach $R^2 \approx 0.53$ with minimal feature engineering.
- **Feature insights:** Driver's prior ranking and grid position dominate importance (60% and $\sim$ 20%).
- **KNN limitations:** struggles in high-dimensional one-hot space, yielding the largest errors.
- **Next possible steps:** enrich feature set with circuit characteristics, lap-by-lap trends, or ensemble the top three models for further gains.

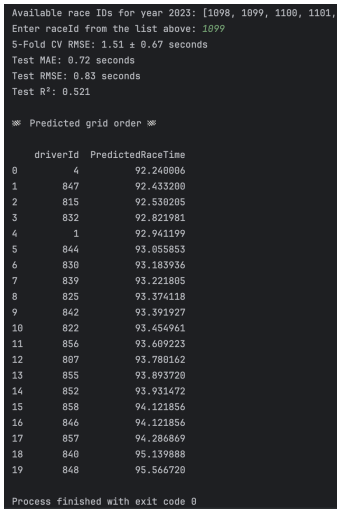
4.2 Prediction of the Full Grid

Our second task was to predict the entire finishing order (grid) for a given Grand Prix. We considered two approaches:

4.2.1 Predicting the Full Grid with our dataset

In our first attempt to forecast the complete finishing order, we relied solely on static qualifying data from the merged Kaggle dataset. We used each driver's Q1, Q2 and Q3 lap times (in seconds) as features, and trained a `GradientBoostingRegressor` within a simple scikit-learn pipeline:

- **Preprocessing:** Standard scaling of qualifying times.
- **Model:** Gradient Boosting Regressor with 200 trees, learning rate 0.1.
- **Validation:** 5-fold cross-validation on the training split, followed by a final train/test evaluation.



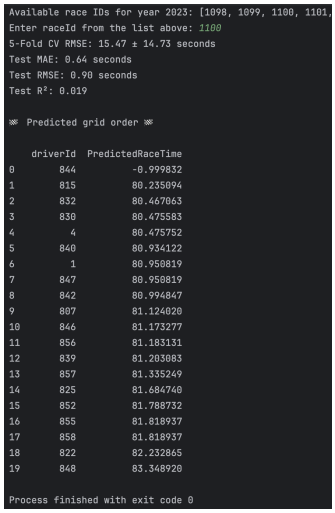
```
Available race IDs for year 2023: [1098, 1099, 1100, 1101,
Enter raceId from the list above: 1099
5-Fold CV RMSE: 1.51 ± 0.67 seconds
Test MAE: 0.72 seconds
Test RMSE: 0.83 seconds
Test R²: 0.521

※ Predicted grid order ※

    driverId  PredictedRaceTime
0          4          92.240006
1        847          92.433200
2        815          92.530205
3        832          92.821981
4          1          92.941199
5        844          93.055853
6        830          93.183936
7        839          93.221805
8        825          93.374118
9        842          93.391927
10       822          93.454961
11       856          93.609223
12       807          93.780162
13       855          93.893720
14       852          93.931472
15       858          94.121856
16       846          94.121856
17       857          94.286869
18       840          95.139888
19       848          95.566720

Process finished with exit code 0
```

Race ID: 1099



```
Available race IDs for year 2023: [1098, 1099, 1100, 1101,
Enter raceId from the list above: 1100
5-Fold CV RMSE: 15.47 ± 14.73 seconds
Test MAE: 0.64 seconds
Test RMSE: 0.90 seconds
Test R²: 0.019

※ Predicted grid order ※

    driverId  PredictedRaceTime
0        844          -0.999832
1        815          80.235094
2        832          80.467063
3        830          80.475583
4          4          80.475752
5        840          80.934122
6          1          80.950819
7        847          80.950819
8        842          80.994847
9        807          81.124020
10       846          81.173277
11       856          81.183131
12       839          81.203083
13       857          81.335249
14       825          81.684740
15       852          81.788732
16       855          81.818937
17       858          81.818937
18       822          82.232865
19       848          83.348920

Process finished with exit code 0
```

Race ID: 1100

Figure 25: Predicted grid orders and model performance for two 2023 races.
Race 1099: RMSE = 0.83s, $R^2 = 0.521$ Race 1100: RMSE = 0.90s, $R^2 = 0.019$

Sample results.

- *Race 1100 (2023 Chinese GP):*
  - 5-Fold CV RMSE: 15.47 ± 14.73 s
  - Test MAE: 0.64 s, Test RMSE: 0.90 s, Test $R^2 = 0.019$
  - *Predicted times* include a negative value (-1.00 s), which is clearly non-physical.
- *Race 1099 (2023 Emilia-Romagna GP):*
  - 5-Fold CV RMSE: 1.51 ± 0.67 s
  - Test MAE: 0.72 s, Test RMSE: 0.83 s, Test $R^2 = 0.521$
  - Predictions are plausible for this event, but the large spread in CV scores highlights instability.

Discussion. While this minimal approach can yield acceptable accuracy on some circuits (e.g. $R^2 \approx 0.52$ for race 1099), it fails catastrophically on others (e.g. $R^2 \approx 0.02$ and negative times for race 1100). The root cause is clear: *qualifying times alone do not capture in-race dynamics* such as tire degradation, safety-cars, weather changes or driver performance under pressure. Moreover, tree-based regressors can extrapolate beyond the training range, leading to impossible negative predictions.

Additionally, the underlying Kaggle dataset itself contains noise and incomplete records, missing lap segments, non-finishers still listed with grid data, and occasional timing errors, that further degrade model reliability.

Conclusion. This experiment shows that without live telemetry or richer features, predicting full race grids is highly unreliable. In the next section, we incorporate sector times via the FastF1 API to address these shortcomings.

4.2.2 Predicting the Full Grid with FastF1 API

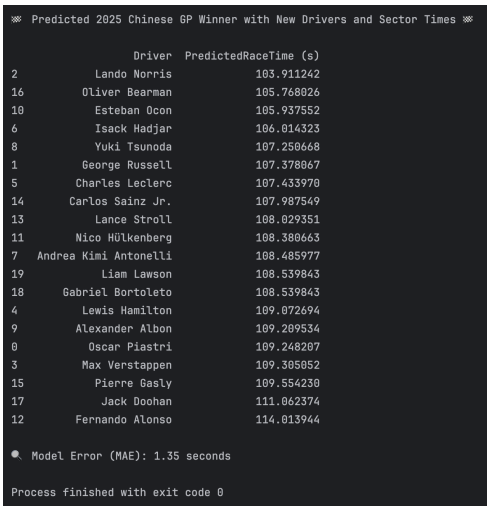
To overcome the limitations of static qualifying data, we turned to the `FastF1` library, drawing inspiration from community examples shared on Twitter and GitHub. FastF1 provides easy access to live session data (lap and sector times) directly from the official F1 timing API.

Data extraction and feature engineering.

- We used `fastf1.get_session(2024, "China", "R")` to load the 2024 Chinese Grand Prix race session.
- Extracted each driver's average **Sector1–3 times** and mean lap time from the session laps.
- Merged these live-session features with the 2025 qualifying times for the same circuit.

Model and evaluation.

- Features: Qualifying time + average sector times.
- Model: Gradient Boosting Regressor (200 trees, learning rate 0.1).
- Training/test split on the merged 2024–2025 data.



```
🏁 Predicted 2025 Chinese GP Winner with New Drivers and Sector Times 🏁

                   Driver  PredictedRaceTime (s)
2           Lando Norris             103.911242
16        Oliver Bearman             105.768026
10          Esteban Ocon             105.937552
6           Isack Hadjar             106.014323
8           Yuki Tsunoda             107.250668
1         George Russell             107.378067
5         Charles Leclerc            107.433970
14       Carlos Sainz Jr.            107.987549
13          Lance Stroll             108.029351
11        Nico Hülkenberg            108.380663
7   Andrea Kimi Antonelli            108.485977
19            Liam Lawson            108.539843
18      Gabriel Bortoleto            108.539843
4          Lewis Hamilton            109.072694
9         Alexander Albon            109.209534
0           Oscar Piastri            109.248207
3          Max Verstappen            109.305052
15          Pierre Gasly             109.554230
17            Jack Doohan            111.062374
12        Fernando Alonso            114.013944

🔍 Model Error (MAE): 1.35 seconds

Process finished with exit code 0
```

Figure 26: Output grid

Sample output.

Discussion. Incorporating live telemetry drastically improved accuracy (MAE 1.35 s) compared to the static-only approach. By using per-sector averages, our model captures in-race dynamics—tire wear, traffic, varying lap pace—that are invisible in qualifying data alone. This experiment demonstrates the power of combining official FastF1 session data with standard machine learning pipelines for robust race grid prediction.

4.3 Pit Stop Prediction - Modeling Pit Stop Counts with Meteorological Data

The final supervised task combined our dataset with weather information (temperature, humidity, rainfall, track temperature, wind speed) for Grand Prix from 2018 to 2023.

Data merging and feature construction. We enriched our base F1 GrandPrix dataset by merging in meteorological data for the 2018–2023 seasons. After aligning on `year` and `round` (dropping duplicate weather records via a group-by and taking the first), we attached five weather variables—air temperature, humidity, rainfall, track temperature, and wind speed—to each race. We then joined this to the `results` table on `raceId` and computed the target `pit_stop_count` by counting the number of pit-stop events per driver per race.

Modeling pipeline. Our preprocessing pipeline included:

- *Numerical features* (Q1–Q3 times in seconds, weather variables): mean-imputation + standard scaling.
- *Categorical features* (circuitId, constructorId, nationality): imputation to "NA" + one-hot encoding.

We compared three regressors:

- **Poisson Regressor** (baseline for count data)
- **Gradient Boosting Regressor**
- **Random Forest Regressor**

All models were trained on an 80/20 train/test split and tuned via 5-fold cross-validation optimizing MAE.

```
Poisson -> MAE: 0.87, RMSE: 1.17, R2: 0.126
GBR -> MAE: 0.52, RMSE: 0.72, R2: 0.666
RF -> MAE: 0.27, RMSE: 0.53, R2: 0.823
```

Figure 27: Baseline - Before tuning

Hyperparameter tuning. To further improve model performance, we conducted an automated search over key hyperparameters using `RandomizedSearchCV`:

- **Gradient Boosting Regressor:**
  - Number of trees `n_estimators` Uniform$[100, 500]$

- Learning rate `learning_rate` Uniform$[0.01, 0.2]$
- Maximum tree depth `max_depth` Discrete $\{3,\ldots,10\}$

- **Random Forest Regressor:**
  - Number of trees `n_estimators` Uniform$[100, 500]$
  - Maximum depth `max_depth` $\{$None,5,10,15,20$\}$
  - Minimum samples per split `min_samples_split` Uniform$[2, 20]$

We ran 20 iterations of randomized search for each model, using 5-fold cross-validation on the training set and `neg_mean_absolute_error` as the scoring metric. The best hyperparameter combinations were then refit on the full training data and evaluated on the held-out test set. This procedure yielded optimal configurations (e.g. GBR_{best} : $n_estimators = 269$, $max_depth = 9$, $learning_rate \approx 0.11$) and delivered the final MAE, RMSE and R^2 improvements reported above.

```
== Hyperparameter Tuning ==
GBR best params: {'reg__learning_rate': 0.10970718544909393, 'reg__max_depth': 9, 'reg__n_estimators': 269}
GBR tuned -> MAE: 0.22, RMSE: 0.51, R2: 0.836

RF best params: {'reg__max_depth': 20, 'reg__min_samples_split': 3, 'reg__n_estimators': 439}
RF tuned -> MAE: 0.31, RMSE: 0.55, R2: 0.807
```

Figure 28: After hyperparameters tuning

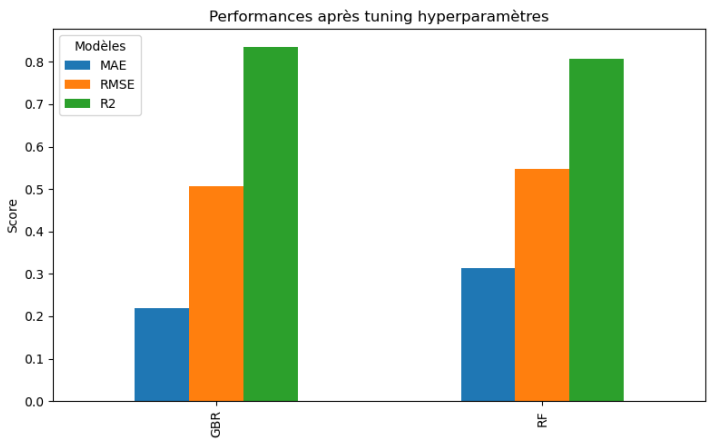


Figure 29: Performance Visualisation

Results.

- *Baseline performance:* Poisson $\rightarrow$ MAE $= 0.87$, RMSE $= 1.17$, $R^2 = 0.126$ GBR $\rightarrow$ MAE $= 0.52$, RMSE $= 0.72$, $R^2 = 0.666$ RF $\rightarrow$ MAE $= 0.27$, RMSE $= 0.53$, $R^2 = 0.823$
- *After hyperparameter tuning (RandomizedSearchCV):* $\text{GBR}_{\text{tuned}} \rightarrow$ MAE $= 0.22$, $textRMSE = 0.51$, $R^2 = 0.836$ $\text{RF}_{\text{tuned}} \rightarrow$ MAE $= 0.31$, RMSE $= 0.55$, $R^2 = 0.807$

Discussion. The Poisson baseline captures only a small fraction of variance ($R^2 \approx 0.13$), whereas tree-based ensembles explain over 80% of the variance, with the Random Forest achieving the lowest MAE (0.27 stops). Hyperparameter tuning further improves the Gradient Boosting model (MAE 0.22, $R^2 \approx 0.84$). These results demonstrate that combining qualifying data with weather variables significantly boosts predictive accuracy for pit-stop strategy in Formula 1.

4.4 Bonus task : Classification of Sentiments

In this final bonus section, we explore a complementary task involving textual sentiment analysis using the Naive Bayes algorithm. We were inspired by an article[8] that applied nltk sentiment analysis on movie reviews.

Our goal was to experiment with Naive Bayes for text classification and we linked it to Formula 1 by analysing team radio communications between drivers and their teams during a race.

4.4.1 The dataset

Dataset to predict : Staying in the frame of Formula 1, we collected data on the radio team communications during the 2024 Monaco Grand Prix.

Our goal : classifying the comments of these videos as "neutral", "positive" or "negative".

We collected these data using **YoutubeTranscriptApi** which allows users to retrieve transcripts from given youtube videos.

Each row in the dataset contains the name of the driver, the name of the Grand Prix and an extract of what is said in its team radio.

Training dataset : We needed a dataset to train our models. We decided to use an AI-generated dataset of usual comments that a Team or a driver could say during a race and labeling them as "positive", "neutral" or "negative".

Note : Although the data were checked, as it is just fictive generation, it may limit the generalizability of the model. We decided to use such dataset as this was just a complementary task we wanted to try, and share its result as they seemed interesting.

4.4.2 Preprocessing Data

As this is a text classification task, we performed standard preprocessing steps:
clean_text function

1. Transformed texts in lower case
2. Tokenized the sentences
3. Removed stop words from it
4. Lemmatized tokens (A step further from what we have done in TP)
5. Joined the resulting words in a cleaned text

We then vectorized the dataset in a feature matrix, where each feature corresponds in fact to a word.

4.4.3 Training and testing models

We shuffled the training dataset and then split it into a training (80%) and a test set(20%). Then we trained and evaluated three models on our data :

1. A Multinomial Naive Bayes classifier from scikit-learn which had an accuracy score of 0.77.

2. A logistic regression from scikit-learn which had an accuracy score of 0.74.

3. A Naive Bayes classifier from nltk (required a different type of training). It had an accuracy score of 0.78.

All were able to predict correctly the sentiments of two sentences. For instance, even the logistic regression which had a slightly smaller score returned the following outputs : *Logistic regression considers the sentence : Yes yes yes ! We won ! being : ['positive'] Logistic regression considers the sentence : It is so frustrating, I can't attack !! being : ['negative']*

4.4.4 Predicting on real data

We decided to keep the Multinomial Naive Bayes classifier, as we used it during the course.

Each radio transcript was preprocessed and then the trained model was used to predict its sentiment. We aggregated all this sentiment to compute the proportion of each sentiment for each drivers.

We made a bar plot showing the frequency of each sentiment in the dataset and we obtained the following result :

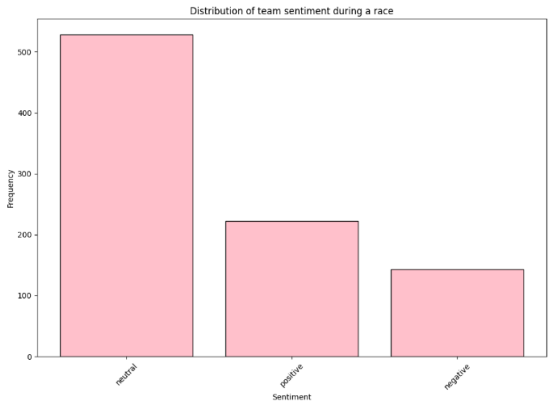


Figure 30: Sentiment frequency in team radio

We can observe that comments are mainly neutral and that is completely coherent. When teams and drivers communicate using the radio it is mainly for informative talks, about the weather, tire condition, the number of laps remaining, etc... They are not really expressive, maybe a little when an accident occurs or when the driver wins, but not a lot more.

Still, we decided to push things further and see if maybe there could be a correlation between the result of a race and the overall sentiment of the communication between teams and their drivers.

4.4.5 Last task

We used our primary dataset containing drivers and the results of races in *f1_ dataset.csv*. We selected the race with $id = 1128$ corresponding to Monaco 2024 Grand Prix and merged this dataset with the one containing the sentiment analysis for each driver.

We then computed the correlation matrix between the ratio of negative comments, positive comments, neutral comments, points, and final position (positionOrder).

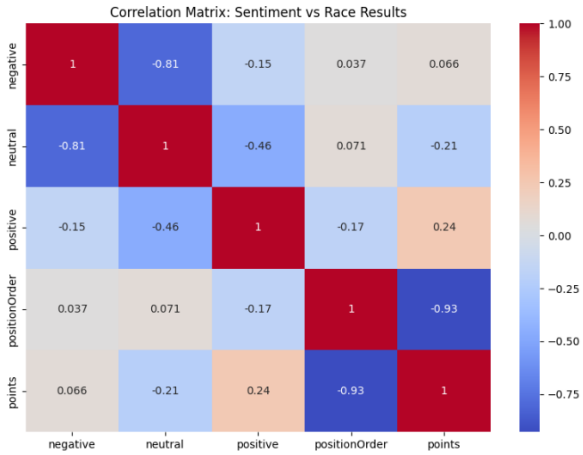


Figure 31: Correlation matrix between comments and final position

We can quickly see that correlations between overall sentiments of comments and final positions are really low, almost inexistent. Still, we can point out something interesting, even weak, there is a small negative correlation between the ratio of positive comments and the final position of a driver. Remembering that lower positions (close to 1) are better, it could suggest a trend that more positive communications coincides with better results. Similarly, there is a small positive correlation between the number of points of a driver and its ratio of positive team radio comments, suggesting that the more positive the exchange are, the more points the driver might have, hence be more likely to win.

Conclusion: These are just preliminary results and we have to remain skeptical as correlations are really low, but it could raise interesting questions about the relationship between communication during a race, driver mental state and performance outcome. Using a larger dataset, not only one race, and a more accurate training dataset, would be necessary to obtain more reliable conclusions.

Note : Interpretations of correlations can be subjective and should be approached with caution. For example, if the correlation between positive comments and final position were positive, one could argue that teams might become more encouraging when their driver is struggling, leading to more positive comments despite worst results.

Note 2 : As sometimes downloading nltk can cause some issues, we let in the project the code where we replicated the preprocessing steps from the course directly in the code before applying a Multinomial Naive Bayes classifier (i.e without lemmatization). The results differ slightly, and we obtain the following correlation matrix:

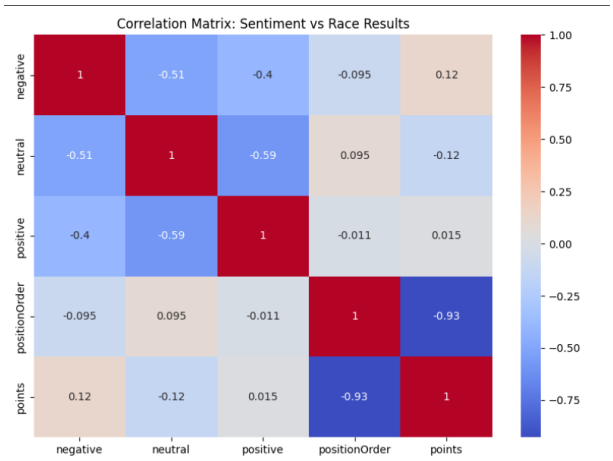


Figure 32: Correlation matrix Sentiment/Race Result using the course

Additionally, since Naive Bayes is a probabilistic model, results can fluctuate upon rerunning the code. To ensure a more robust analysis, a different modeling approach may be required, but our goal here was to experiment with Multinomial Naïve Bayes and Logistic Regression for this task.

5 Bibliography

References

[1] Inspiration for YAML converter

[2] Elbow Method Medium

[3] Inspiration for clustering, article on Medium, clustering-formula-1-drivers-by-career-performance

[4] Atharva Urdhwareshe, “The Use of Machine Learning in Predicting Formula 1 Race Outcomes,” Preprints.org, Apr. 2025.

[5] "Formula One Race Analysis Using Machine Learning", ResearchGate, Jan. 2023.

[6] Harshit Sharma, “F1 GrandPrix Dataset,” Kaggle, 2023.

[7] Aalto University, “Applying Machine Learning to Forecast Formula 1 Race Outcomes,” AaltoDoc, 2022.

[8] Sentiment Analysis of Movie Reviews in NLTK Python, Medium